

1. PROJECT COVER

POCKET SMART AI: YOUR SMART BUDGET

Naan Mudhalvan Project

Sub Title: Define API Routes

Project Theme

An intelligent personal budgeting platform that helps users understand income, plan expenses, monitor spending, and make better day-to-day financial decisions through a simple digital experience.

Prepared as an academic project for Naan Mudhalvan

Student Name: _____

Register Number: _____

Department / Institution: _____

Academic Year: 2026–2027

2. ABSTRACT

Pocket Smart AI: Your Smart Budget is a proposed personal finance application designed to make budgeting easier for students and everyday users. The system combines a clean interface with structured financial data and API-based services. Users can record income and expenses, categorize transactions, define spending limits, review summaries, and receive useful budget insights.

The project focuses on a backend architecture in which clearly defined API routes connect the user interface, business logic, and database. This approach supports modular development, easier testing, data validation, authentication, and future integration with analytics or AI-based recommendation services.

The central objective is to present a user's own financial information in a clear and actionable form. The solution can be extended with dashboards, alerts, monthly reports, savings goals, recurring expenses, and intelligent spending-pattern analysis.

3. INTRODUCTION

Managing personal money can be difficult when expenses are frequent, small, and spread across multiple categories. Students and young adults may know their income but still struggle to understand where their money goes. A digital budget planner can solve this problem by bringing transactions, limits, goals, and summaries into one place.

Pocket Smart AI is designed around the idea of a compact financial assistant. The application records financial activity and transforms raw transaction data into understandable information. Users can see current spending, compare it with planned limits, and identify categories that need attention.

The project also demonstrates practical software-development skills such as REST API design, database modelling, validation, authentication, error handling, and frontend-backend integration.

4. PROBLEM STATEMENT

Many personal budgeting methods depend on notebooks, spreadsheets, memory, or disconnected applications. These methods can become difficult to maintain when transactions increase. Users need a simple way to capture financial activity and receive an organized view of their budget.

Key problems:

- Difficulty tracking daily expenses consistently.
- Lack of a single view of income, expenses, and remaining budget.
- Limited awareness of category-wise spending.
- Manual calculations can lead to mistakes.
- Users may not receive timely reminders when a budget limit is close.
- Data is often not structured for future analytics.

5. OBJECTIVES

The main objective is to develop a reliable and user-friendly budgeting system with a clearly defined API layer.

1. Create secure user registration and authentication.
2. Add, edit, view, and delete transactions.
3. Organize expenses into categories.
4. Define monthly and category-based budgets.
5. Calculate income, expense, and balance summaries automatically.
6. Provide dashboard data through API routes.
7. Validate inputs and return consistent responses.
8. Keep the architecture ready for AI-assisted spending insights.
9. Provide a maintainable database structure.
10. Test API routes for correctness and reliability.

6. EXISTING SYSTEM & PROPOSED SYSTEM

Existing approach: Users commonly depend on manual notes, spreadsheets, calculator-based tracking, or multiple disconnected tools. These approaches require repeated data entry and may not provide a unified dashboard.

Proposed approach: Pocket Smart AI centralizes financial records. A frontend communicates with backend API routes. The backend validates requests, applies business rules, stores data, and returns structured responses.

Area	Existing Approach	Proposed System
Tracking	Manual / scattered	Centralized digital records
Calculations	Manual	Automatic summaries
Budgets	Static notes	API-managed limits
Reports	Manual	Generated from stored data
Future AI	Limited	Architecture-ready

7. SYSTEM REQUIREMENTS

Functional requirements

- User registration and login
- Profile management
- Income and expense entry
- Category management
- Budget creation and update
- Transaction search and filtering
- Dashboard summary
- Monthly reports
- Budget alerts
- Optional AI-based insights

Non-functional requirements

The system should be secure, responsive, maintainable, scalable, and easy to use. API responses should be predictable and validation errors should be understandable. Sensitive information should be protected using appropriate authentication and secure storage practices.

8. TECHNOLOGY STACK

The project can be implemented using a modern web application stack.

Layer	Suggested Technology	Purpose
Frontend	HTML/CSS/JavaScript or React	User interface
Backend	Node.js + Express	REST API server
Database	MongoDB / MySQL	Financial data storage
Authentication	JWT / session-based auth	User access control
Testing	Postman / automated tests	API validation
Version Control	Git	Source-code management
Deployment	Cloud platform	Application hosting

9. SYSTEM ARCHITECTURE

The application follows a layered architecture. The frontend sends HTTP requests to API routes. The API layer validates authentication and input data before passing the request to business logic. The service layer performs calculations and rules, while the data-access layer communicates with the database.

Flow: User → Web/Mobile UI → API Routes → Controllers → Services → Database → API Response → UI

This separation improves maintainability because each component has a focused responsibility. Budget calculations can be tested independently from the user interface.

Security principle: Authentication tokens, passwords, and financial data should be handled securely. Passwords must never be stored as plain text.

10. DATABASE DESIGN

A relational or document database can be used depending on implementation preference. The following logical entities provide a useful starting point.

Entity	Important Fields
User	userId, name, email, passwordHash, createdAt
Transaction	transactionId, userId, type, amount, categoryId, note, date
Category	categoryId, userId, name, type
Budget	budgetId, userId, categoryId, limit, month
Goal	goalId, userId, name, targetAmount, savedAmount, targetDate

11. DEFINE API ROUTES

The API route layer is the core technical focus of this project. Routes should use clear HTTP methods and consistent resource names.

Method	Route	Purpose
POST	/api/auth/register	Create user account
POST	/api/auth/login	Authenticate user
GET	/api/users/me	Get current user
GET	/api/transactions	List transactions
POST	/api/transactions	Create transaction
PUT	/api/transactions/:id	Update transaction
DELETE	/api/transactions/:id	Delete transaction
GET	/api/categories	List categories
POST	/api/categories	Create category
GET	/api/budgets	List budgets
POST	/api/budgets	Create budget
PUT	/api/budgets/:id	Update budget
GET	/api/dashboard	Get summary data

12. API REQUEST & RESPONSE DESIGN

A consistent API contract makes the application easier to develop and test. Every request should define required fields, authentication requirements, validation rules, and expected status codes.

Example: Create Transaction

POST /api/transactions

Request: type, amount, categoryId, note, date

Success: HTTP 201 with transaction identifier and stored transaction details.

Validation failure: HTTP 400 with field-level error information.

Unauthenticated: HTTP 401.

Example response:

```
{ "success": true, "data": { "transactionId": "TXN001", "amount": 250, "type": "expense" }, "message":  
"Transaction created successfully" }
```

13. AUTHENTICATION & SECURITY

Security is important because the application handles personal financial information. The authentication layer should verify identity before protected routes are accessed.

Recommended controls include password hashing, secure token handling, authorization checks, input validation, rate limiting where appropriate, HTTPS in production, safe error messages, and protected database access. Users should only be able to read or modify records belonging to their own account.

API routes should reject malformed requests and should not expose sensitive implementation details. Database queries should use safe parameterization or trusted ORM/database APIs to reduce injection risks.

14. BUDGET LOGIC

The application can calculate the current financial position using simple, transparent formulas.

Total Income = Sum of all income transactions in the selected period.

Total Expenses = Sum of all expense transactions in the selected period.

Balance = Total Income – Total Expenses.

Budget Remaining = Category Budget Limit – Category Expenses.

The dashboard can show these values together with category-level spending. When a category approaches its configured limit, the system can generate an alert. Thresholds should be configurable rather than assumed to be suitable for every user.

15. AI-ASSISTED SMART INSIGHTS

The 'Smart' part of Pocket Smart AI can be implemented as an optional analysis layer over user-authorized transaction data. The system can identify patterns such as unusually high spending in a category, repeated expenses, or changes between months.

Examples include: 'Food spending is higher this month than last month' or 'You are close to your transport budget limit.' Such messages should be based on actual stored data and presented as informational insights.

A future version may include natural-language queries such as 'How much did I spend on food this month?' The AI component should retrieve authorized data through controlled services rather than directly accessing the database without validation.

16. USER INTERFACE & DASHBOARD

The dashboard should prioritize information that users need quickly. A suggested layout contains a balance card, income card, expense card, budget progress, recent transactions, category breakdown, and smart insights.

Main screens:

1. Login / Register
2. Dashboard
3. Add Transaction
4. Transactions List
5. Budget Manager
6. Categories
7. Savings Goals
8. Monthly Report
9. Profile and Settings

The interface should be responsive so that the same core features work on mobile and desktop screens.

17. TESTING PLAN

Testing verifies that API routes behave correctly for valid and invalid inputs.

Test Case	Expected Result
Valid registration	Account created
Duplicate email	Validation error
Valid login	Authentication succeeds
Invalid password	Authentication rejected
Create transaction	HTTP 201
Invalid amount	Validation rejected
Access another user's record	Authorization rejected
Update own budget	Budget updated
Dashboard request	Correct summary returned
Delete transaction	Record removed or marked deleted

18. IMPLEMENTATION ROADMAP

Phase 1 — Planning: finalize requirements, user flow, database entities, and API contracts.

Phase 2 — Backend: create project structure, database connection, authentication, models, controllers, services, and routes.

Phase 3 — Frontend: build dashboard, forms, transaction pages, budget pages, and reports.

Phase 4 — Integration: connect frontend to APIs and handle loading, success, and error states.

Phase 5 — Testing: perform API, validation, authorization, and usability testing.

Phase 6 — Deployment: configure production environment, secure secrets, database access, monitoring, and backups.

19. EXPECTED OUTCOMES & FUTURE SCOPE

The completed project is expected to provide a practical budgeting workflow in which users can record transactions, define budgets, review financial summaries, and understand spending patterns.

Future scope:

- Expense prediction using historical patterns
- Voice-based expense entry
- Receipt scanning and OCR
- Custom financial goals
- Recurring transaction automation
- Exportable monthly reports
- Multi-device synchronization
- Advanced analytics and visualization
- Configurable alerts

Future features should be added incrementally, with privacy, security, data accuracy, and user control treated as core requirements.

20. CONCLUSION & REFERENCES

Pocket Smart AI: Your Smart Budget presents a practical approach to personal budgeting using a modern API-driven architecture. The project demonstrates how structured data, REST API routes, authentication, database design, validation, and a responsive interface can work together to create a useful financial management application.

The 'Define API Routes' component provides a strong technical foundation because it creates a clear contract between the frontend and backend. With additional testing and refinement, the project can be extended into a complete budgeting application with intelligent, data-driven insights.

References / Learning Resources

- REST API design principles and HTTP methods
- Node.js and Express documentation
- Database design concepts
- JSON data interchange standards
- Web application security and authentication practices
- Naan Mudhalvan course/project materials

Project motto: Track wisely. Plan clearly. Spend confidently.